Chapter five

Intermediate Code Generation

A source code can directly be translated into its target machine code, then why at all we need to translate the source code into an intermediate code which is then translated to its target code?

✓ In the analysis-synthesis model of a compiler, the front end translates a source program into an intermediate representation from which backend generates target code.



Fig. Intermediate code generation in compiler

Let us see the reasons why we need an intermediate code.

- If a compiler translates the source language to its target machine language without having the option for generating intermediate code, then for each new machine, a full native compiler is required.
- Intermediate code eliminates the need of a new full compiler for every unique machine by keeping the analysis portion same for all the compilers.
- The second part of compiler, synthesis, is changed according to the target machine.
- It becomes easier to apply the source code modifications to improve code performance by applying code optimization techniques on the intermediate code.

Intermediate Representation

Intermediate codes can be represented in a variety of ways and they have their own benefits.

- **High Level IR** - High-level intermediate code representation is very close to the source language itself. They can be easily generated from the source code and we can easily apply code modifications to enhance performance. But for target machine optimization, it is less preferred.
- **Low Level IR** - This one is close to the target machine code, which makes it suitable for register and memory allocation, instruction set selection, etc. It is good for machine-dependent optimizations. Intermediate code can be either language specific (e.g., Byte Code for Java) or language independent (three-address code).

Intermediate code representation can be done in:

1. **Postfix Notation**
2. **Three-address Code**

However, Three-Address Code representation technique is more efficient one.

1. **Postfix Notation**

Postfix notation is the notation in which operators are placed after the corresponding operands in the expression. For example the expression **a + b** (which is infix) is represented in postfix notation, as **a b +.** This notation takes less time for parsing. The revers of this is known as prefix notation, which is + **a b.** It is a way of writing algebraic expressions without the use of parentheses or rules of operator precedence.

- **Rules to represent postfix notations**
  1. Consider operator priorities. Exponent has highest , /* next and +- lowest priority
  2. No two operators of same priority can stay together on stack column. E.g. (+ -) and (/ *) have same priority and cannot stay together . If it happens pop-top of stack operator before push next operator.
  3. Lowest priority operator cannot placed before highest priority operator. If it happens, pop highest priority operator from stack first and push lowest priority operator to postfix arrangement.
  4. If the operator is enclosed by ( ) then pop the operator to postfix.

  Example (A+B/C*(D+E) –F) . This is infix, let us convert to postfix notation. When we scan the given expression from left to right, if it is operand placed on postfix and if it is operator, put on top of stack.

Symbol	stack	postfix
(	(	
A	(	A
+	(+	A
B	(+	AB
/	(+/	AB here + is lower than / so no need to pop +
C	(+/	ABC
*	(+*	ABC/ because / * cannot stay together. / is popped and place in postfix
(	(+*(	ABC/
D	(+*(	ABC/D
+	(+*(+	ABC/D
E	(+*(+	ABC/DE
)	(+*	ABC/DE+ (+) is popped and placed on postfix
-	(-	ABC/DE+*+ * is higher than – so pop * and again – and + can't stay together pop +
F	(-	ABC/DE+*+F
)	(-)	ABC/DE+*+F-

Thus the expression **(A+B/C*(D+E) –F)** is converted to **ABC/DE+*+F-** in postfix notation.

Three-Address Code

Intermediate code generator receives input from its predecessor phase, semantic analyzer, in the form of an annotated syntax tree. That syntax tree then can be converted into a linear representation, e.g., postfix notation. Intermediate code tends to be machine independent code. Therefore, code generator assumes to have unlimited number of memory storage (register) to generate code.

- ✓ Three address code is a sequence of statements of the general form,

  a = b op c

- ✓ Where a, b or c are the operands that can be names or constants and op stands for any operator.
- Example: a = b + c + d

  Three address code for above expression is:

  $t_1 = b+c$

  $t_2 = t_1+d$

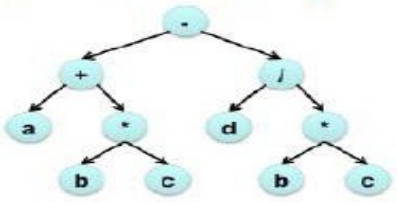
  $a = t_2$

- ✓ Here t_1 and t_2 are the temporary names generated by the compiler.
- ✓ Only single operator on the right side of the expression is allowed at a time.
- ✓ There are at most three addresses allowed (two for operands and one for result). Hence, this representation is called three-address code.

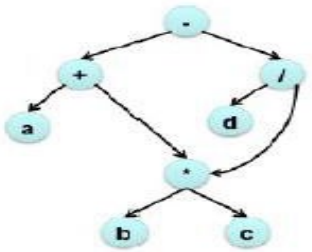
Example 2

IR Code is Made From

- (a+b*c) – (d/(b*c))



Syntax tree



DAG

- Intermediate Code

1. t1 = b*c
2. t2 = a+t1
3. t3 = b*c
4. t4 = d/t3
5. t5 = t2-t4

t1 = b * c
t2 = a + t1
t3 = d / t1
t4 = t2 – t3

Three-Address Code IR can be implemented by three ways:

1. Quadruple
2. Triple
3. Indirect Tipple

1. Quadruple

- ✓ The quadruple is a structure with at the most four fields such as op, arg1, arg2 and result.
- ✓ The op field is used to represent the internal node for operator. The arg1 and arg2 represent the two operands. And result field is used to store the result of an expression.
- ✓ The three address statements with unary operator like a = -y or a = y do not use agr2.

Eg X= -a*b + -a*b

Pointer's in the symbol table

t_1= uminus a
$t_2 = t_1 * b$
t_3= uminus a
$t_4 = t_3 * b$
$t_5 = t_2 + t_4$
x= t_5

Intermediate Code

Quadruple representation

	Operator	Arg1	Arg2	Result
(0)	uminus	a		t_1
(1)	*	t_1	b	t_2
(2)	uminus	a		t_3
(3)	*	t_3	b	t_4
(4)	+	t_2	t_4	t_5
(5)	=	t_5		x

2. Triple

- ✓ In triple, temporaries are not used instead of that pointers in the symbol table are used directly
- ✓ If we do so, three address statements can be represented by records with only three fields: op, arg1 and arg2.
- ✓ Numbers in the round bracket ( ) are used to represent pointers into the triple structure.

Quadruple representation

	Operator	Arg1	Arg2	Result
(0)	uminus	a		t_1
(1)	*	t_1	b	t_2
(2)	uminus	a		t_3
(3)	*	t_3	b	t_4
(4)	+	t_2	t_4	t_5
(5)	=	t_5		x

Triple representation

	Operator	Arg1	Arg2
(0)	uminus	a	
(1)	*	(0)	b
(2)	uminus	a	
(3)	*	(2)	b
(4)	+	(1)	(3)
(5)	=	x	(4)

3. Indirect Triples

- ✓ In the indirect triple representation the listing of triples has been done. And listing pointers are used instead of using statement.
- ✓ This implementation is called indirect triples.

Triple representation

No.	Operator	Arg1	Arg2
(0)	uminus	a	
(1)	*	(0)	b
(2)	uminus	a	
(3)	*	(2)	b
(4)	+	(1)	(3)
(5)	=	x	(4)

Indirect Triple representation

	Statement
(0)	(14)
(1)	(15)
(2)	(16)
(3)	(17)
(4)	(18)
(5)	(19)

No.	Operator	Arg1	Arg2
(0)	uminus	a	
(1)	*	(14)	b
(2)	uminus	a	
(3)	*	(16)	b
(4)	+	(15)	(17)
(5)	=	x	(18)

❖ Declarations

A declaration in a program refers to a statement that provides the data about the name and type of data objects to the programming language translators. **For example int a, b;**

This declaration provides the programming language translator with the information that a and b are the data objects of type integer that are needed during the execution of the subprogram.

For every local name in a procedure, we create a ST (Symbol Table) entry containing:

1. The type of the name
2. How much storage the name requires

The production:

1. D → integer, id
2. D → real, id
3. D → D1, id

A suitable transition scheme for declarations would be:

Production rule	Semantic action
D → integer, id	ENTER (id.PLACE,integer) , D.ATTR = integer
D → real, id	ENTER (id.PLACE, real) , D.ATTR = real
D → D1, id	ENTER (id.PLACE, D1.ATTR) D.ATTR = D1.ATTR

ENTER is used to make the entry into symbol table and **ATTR** is used to trace the data type

❖ Procedures call

Procedure is an important and frequently used programming construct for a compiler. It is used to generate good code for procedure calls and returns.

❖ Calling sequence:

The translation for a call includes a sequence of actions taken on entry and exit from each procedure. Following actions take place in a calling sequence:

- ✓ When a procedure call occurs then space is allocated for activation record.
- ✓ Evaluate the argument of the called procedure.
- ✓ Establish the environment pointers to enable the called procedure to access data in enclosing blocks.
- ✓ Save the state of the calling procedure so that it can resume execution after the call.
- ✓ Also save the return address of the location to which the called routine must transfer after it is finished.
- ✓ Finally generate a jump to the beginning of the code for the called procedure.

Let us consider a grammar for a simple procedure call statement

S → call id(Elist)

Elist → Elist, E

Elist → E

A suitable transition scheme for procedure call would be:

Production Rule	Semantic Action
S → call id(Elist)	for each item p on QUEUE do GEN (param p) GEN (call id.PLACE)
Elist → Elist, E	append E.PLACE to the end of QUEUE
Elist → E	initialize QUEUE to contain only E.PLACE

Queue is used to store the list of parameters in the procedure call.

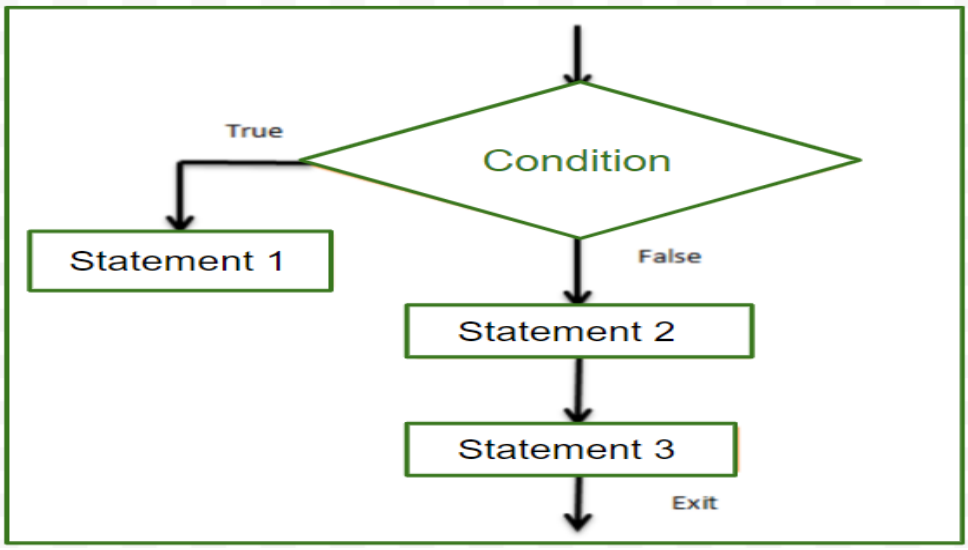
❖ **Back Patching**

Backpatching is basically a process of fulfilling unspecified information. This information is of labels. It basically uses the appropriate semantic actions during the process of code generation. It may indicate the address of the Label in goto statements while producing TAC(Three Address Code) for the given expressions. Here basically two passes are used because assigning the positions of these label statements in one pass is quite challenging. It can leave these addresses unidentified in the first pass and then populate them in the second round. Backpatching is the process of filling up gaps in incomplete transformations and information.

Need for Backpatching:

Backpatching is mainly used for two purposes:

*1. **Boolean expression:***
Boolean expressions are statements whose results can be either true or false.
*2. **Flow of control statements:***
The flow of control statements needs to be controlled during the execution of statements in a program. For example:



3. Labels and Gotos:

The most elementary programming language construct for changing the flow of control in a program is a label and ***goto***. When a compiler encounters a statement like ***goto L***, it must check that there is exactly one statement with label ***L*** in the scope of this ***goto*** statement. If the label has already appeared, then the symbol table will have an entry giving the compiler-generated label for the first three-address instruction associated

with the source statement labeled ***L***. For the translation, we generate a ***goto*** three-address statement with that compiler-generated label as a target.

When a label **L** is encountered for the first time in the source program, either in a declaration or as the target of the forward goto, we enter **L** into the symbol table and generate a symbolic table for **L**.

One-pass code generation using backpatching:

In a single pass, backpatching may be used to create a boolean expressions program as well as the flow of control statements. The synthesized properties **true list** and **false list** of non-terminal B are used to handle labels in jumping code for Boolean statements. The label to which control should go if B is true should be added to **B.truelist**, which is a list of a jump or conditional jump instructions. **B**.**falselist** is the list of instructions that eventually get the label to which control is assigned when B is false. The jumps to true and false exist, as well as the label field, are left blank when the program is generated for B. The lists **B.truelist** and **B.falselist**, respectively, contain these early jumps.

A statement S, for example, has a synthesized attribute S.nextlist, which indicates a list of jumps to the instruction immediately after the code for S. It can generate instructions into an instruction array, with labels serving as indexes. We utilize three functions to modify the list of jumps:

- **Makelist (i):** Create a new list including only i, an index into the array of instructions and the makelist also returns a pointer to the newly generated list.
- **Merge(p1,p2):** Concatenates the lists pointed to by p1, and p2 and returns a pointer to the concatenated list.
- **Backpatch (p, i):** Inserts i as the target label for each of the instructions on the record pointed to by p.

Backpatching for Boolean Expressions:

Using a translation technique, it can create code for Boolean expressions during bottom-up parsing. In grammar, a non-terminal marker M creates a semantic action that picks up the index of the next instruction to be created at the proper time.

For Example, Backpatching using boolean expressions production rules table:

Step 1: Generation of the production table

Production Rule		Semantic action
E	E_1 OR M E_2	{backpatch (E_1.flist, M.quad); E.Tlist:=merge(E_1.Tlist, E_2.Tlist) E.Flist:= E_2.Flist}
E	E_1 AND M E_2	{backpatch (E_1.Tlist, M.quad); E.Tlist:=E_2.Tlist; E.Flist:=merge(E_1.Flist,E_2.Flist);}
E	NOT E_1	{E.Tlist:=E_1.Flist; E.Flist:=E_1.Tlist;}
E	(E_1)	{E.Tlist:=E_1.Tlist; E.Flist:=E_1.Flist;}
E	id1 relop id2	{E.Tlist:=mklist(nextstate); E.Flist:=mklist(nextstate+); Append('if' id1.place relop.op id2.place 'goto_'); Append('goto_')}
E	true	{E.Tlist:=mklist(nextstate); Append('goto_');}
E	false	{E.Flist:=mklist(nextstate); Append('goto_');}
M	ε	{m.quad:=nextquad;}

Production Table for Backpatching

Step 2: We have to find the TAC(Three address code) for the given expression using backpatching:

(A < B) OR (C < D) AND (P < Q)

10. if (A<B) then goto _____ 1 (True)
11. goto _____ 0 (False)
12. if (C<D) then goto _____ 1(True)
13. goto _____ 0 (False)
14 if (p<q) then goto _____ 1 (True)
15 goto _____ 0 (False)

We have two Operators (Relational <) and (Logical(Boolean) AND and OR) operators.

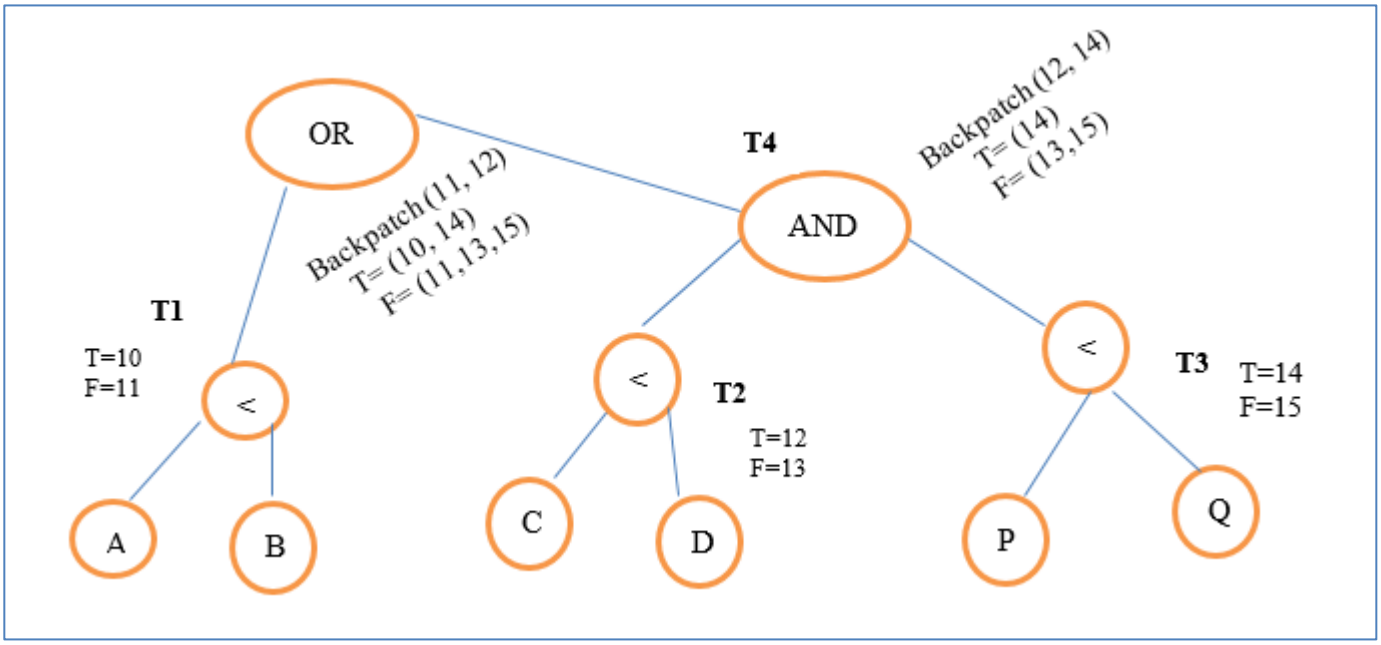
AND operator has highest priority than OR operator.

And let us represent A<B result in T1 , C<D result in T2 and p<q result in T3.

Then (**A < B) OR (C < D) AND (P < Q**) will become **T1** OR **T2** AND **T3.**

And let we store **T2 AND T3** result in **T4**. Then **T1 OR T4** in **T5** .

Step 3: Now us will make the parse tree for the expression:



Now AND operation should be done first on T2 and T3.

Truth table

T2	T3	AND
0	0	0
0	1	0
1	0	0
1	1	1

- When T2 is 0 the result is 0 so no need to check T3 values
- When T2 is 1 the result depends on T3 value. Hence backpatching is required to check T3 value. so when T2 is 1→ True, we have to **backpatch** at AND operation
  As **Backpatch** (**where we get result**, **next quadruple**)
  Which is **Backpatch(12,14)** here 12 is True value of T2 and **13** is False value T2 next quadruple is 14.
- To fined True value of AND, we have to merge True values of T2 and T3. (12 and 14) but 12 is back patched so remove it. → T=(14)
- To fined False value again merge False values of T2 and T3. → F=(13,15)

Now let us Operate OR operation on T1 and T4

T1	T4	OR
0	0	0
0	1	1
1	0	1
1	1	1

- When T1 is 1 the result is 1 so no need to check T4 values. But When T1 is 0 the result depends on T4 value. Hence backpatching is required to check T4 value. so when T1 is 0→ **False** we have to **backpatch** at OR operation
  As **Backpatch** (**where we get result**, **next quadruple**)
  Which is **Backpatch(11,12)** here 11 is 0 (False) value of T1 and **12** is next quadruple.
- To fined True value of OR, we have to merge True values of T1 and T4. (10 and 14)
- To fined False value again merge False values of T1 and T4. → F=(11,13,15)

- **Then when we fill the unspecified fields**

(A < B) OR (C < D) AND (P < Q)

10. if (A<B) then goto _____ 1 (True)
11. goto _12_____ 0 (False)
12. if (C<D) then goto ___14___ 1(True)
13. goto _____ 0 (False)
14 if (p<q) then goto _____ 1 (True)
15 goto _____ 0 (False)

The flow of Control Statements:

Control statements are those that alter the order in which statements are executed. If, If-else, Switch-Case, and while-do statements are examples. Boolean expressions are often used in computer languages to

- **Alter the flow of control:** Boolean expressions are conditional expressions that change the flow of control in a statement. The value of such a Boolean statement is implicit in the program's position. For example, if (A) B, the expression A must be true if statement B is reached.
- **Compute logical values:** During bottom-up parsing, it may generate code for Boolean statements via a translation mechanism. A non-terminal marker M in the grammar establishes a semantic action that takes the index of the following instruction to be formed at the appropriate moment.

Applications of Backpatching:

- Backpatching is used to translate flow-of-control statements in one pass itself.
- Backpatching is used for producing quadruples for boolean expressions during bottom-up parsing.
- It is the activity of filling up unspecified information of labels during the code generation process.
- It helps to resolve forward branches that have been planted in the code.